

POLITIQUE CYBERSÉCURITÉ

Architecture Zero Trust · Security by Design · Local-First · Gouvernance

Cognitive Products Lab · ALFRED · V1.0 · Mai 2026 · CONFIDENTIEL INTERNE

STATUT ET PORTÉE DE CE DOCUMENT

Ce document constitue la politique cybersécurité officielle du projet ALFRED. Il s'applique à tous les composants techniques, à tous les collaborateurs et prestataires intervenant sur le projet, et à toutes les versions (V1 prototype → V6+ production).

Version : V1.0 — Prototype. Ce document sera mis à jour à chaque version majeure.

Responsable : Céline ROUSSELOT, Fondatrice — Cognitive Products Lab.

1. Vision et principes fondateurs

1.1 Déclaration d'intention

La cybersécurité n'est pas une fonctionnalité d'ALFRED. C'est son fondement. Chaque ligne de code, chaque décision d'architecture, chaque interaction utilisateur est conçue avec la sécurité comme contrainte primaire — non comme contrainte secondaire ajoutée après développement.

ALFRED traite des données personnelles sensibles : mémoire conversationnelle, données émotionnelles, données de santé (ARTHUR), commandes vocales, contexte personnel. Cette responsabilité impose un niveau d'exigence maximal.

1.2 Les 8 principes non négociables

Principe	Définition	Application ALFRED
Local-First	Les données sont traitées et stockées localement par défaut.	Aucune donnée utilisateur transmise vers un serveur cloud sans consentement explicite.
Zero Trust	Aucune confiance implicite — tout accès est vérifié, même en interne.	Pipeline : validation → détection → contrôle → auth → MFA → RBAC → politique.
Security by Design	La sécurité est intégrée dès la conception, pas ajoutée a posteriori.	Chaque nouveau module intègre validation, logging et contrôle d'accès dès le premier commit.
Privacy by Design	La protection des données personnelles est intégrée à l'architecture.	Minimisation des données, chiffrement systématique, durées de rétention définies.
Fail-Safe	En cas de doute ou d'erreur, le système refuse par défaut.	Toute requête non validée est rejetée. Silence = refus.

Least Privilege	Chaque composant n'a accès qu'aux ressources strictement nécessaires.	7 rôles RBAC. Aucun module n'a de permissions non justifiées.
Defense in Depth	Multiplication des couches de protection indépendantes.	Input → Threat → Device → Auth → MFA → RBAC → Policy → Output : 8 couches.
Auditabilité	Toute action significative est tracée et consultable.	Audit trail JSONL UTC. Logs sécurité. Dashboard temps réel.

2. Architecture de sécurité

2.1 Pipeline Zero Trust — Description couche par couche

Chaque requête utilisateur traverse obligatoirement les couches suivantes, dans cet ordre. Un refus à n'importe quelle couche arrête le traitement.

#	Couche	Module principal	Contrôles	Latence
1	Validation entrée	input_validator.py	25 patterns : SQLi, XSS, path traversal, cmd injection, prompt injection, SSRF, LDAP, null bytes	Immédiat
2	Détection menace	threat_detector.py + behavioral_detector.py	Score de menace calculé. Analyse comportementale. Seuil configurable.	< 10ms
3	Contrôle appareil	device_registry.py	Trust score appareil. Appareils non reconnus rejetés.	< 5ms
4	Authentification	authenticator.py	bcrypt rounds=12. PIN local. Gestion sessions. Anti brute-force.	< 50ms
5	MFA	mfa_manager.py	TOTP RFC 6238. MFA appareil. MFA comportemental. Secrets chiffrés.	< 30ms
6	RBAC	role_manager.py + permission_manager.py	7 rôles : OWNER / ADMIN / USER / GUEST / SERVICE / AI_MODULE / EMERGENCY	< 5ms
7	Politique Zero Trust	policy_engine.py + zero_trust_orchestrator.py	PDP / PEP. Décision finale. Refus par défaut.	< 10ms
8	Filtrage sortie	output_filter.py	Masquage : clés API, JWT, passwords, tokens, secrets.	< 5ms
9	Audit trail	audit_trail.py + security_logger.py	Journalisation JSONL UTC. Horodatage. Immuable.	Asynchrone

3. Règles de développement sécurisé (Secure SDLC)

3.1 Règles obligatoires — Code

RÈGLES ABSOLUES — TOUT DÉVELOPPEUR DOIT RESPECTER
JAMAIS de secret en clair dans le code (clés API, mots de passe, tokens). → Utiliser .env + python-dotenv.
JAMAIS de .env dans Git. Le .gitignore doit exclure : .env, *.key, data/, knowledges/, *.json sensible.
JAMAIS de except Exception: sans logging. Toute exception doit être loggée via security_logger.
JAMAIS de permission accordée sans validation RBAC. Toute fonction sensible vérifie has_access().
JAMAIS de donnée persistante non chiffrée. Toute écriture dans data/ passe par encryption_service.py.
TOUJOURS valider les inputs via input_validator.py avant tout traitement.
TOUJOURS écrire un test unitaire pour chaque fonction de sécurité.

3.2 Processus de développement sécurisé

Phase	Exigences sécurité
Conception	Modélisation des menaces STRIDE. Revue d'architecture avant développement.
Développement	Secure coding rules. Black + Flake8. Docstrings sécurité.
Test	Tests unitaires sécurité obligatoires. Couverture > 80%. Tests injection.
Revue	Code review axée sécurité avant merge. Vérification .gitignore.
Déploiement	Vérification .env. Validation pipeline Zero Trust. Backup avant déploiement.
Maintenance	Rotation des clés (V2). Mise à jour dépendances. Audit mensuel.

4. Gestion des identités et des accès

4.1 Politique de mots de passe et PIN

Règle	Exigence
Longueur PIN	Minimum 6 chiffres. Recommandé : 8 chiffres ou plus.
Hachage	bcrypt avec rounds=12 minimum. Jamais MD5, SHA1, SHA256 seul.
Tentatives	Blocage automatique après 5 tentatives échouées.
Durée session	Expiration automatique après 900 secondes (15 minutes) d'inactivité.
Changement	Rotation recommandée tous les 90 jours. Obligatoire après incident P1/P2.
MFA	Obligatoire pour les rôles OWNER et ADMIN. Recommandé pour USER.

4.2 Gestion des rôles RBAC

Rôle	Permissions	Règles d'attribution
------	-------------	----------------------

OWNER	Accès total. Administration sécurité. Configuration système.	Réservé à la fondatrice uniquement. Non délégable.
ADMIN	Gestion utilisateurs. Configuration modules. Accès logs.	Attribution par OWNER uniquement. Maximum 2 personnes.
USER	Interaction ALFRED. Lecture mémoire propre. Actions standard.	Utilisateur final standard.
GUEST	Lecture seule. Pas d'écriture mémoire. Sessions limitées.	Accès temporaire. Durée max : 24h.
SERVICE	Accès API inter-modules. Permissions restreintes.	Uniquement pour les modules internes ALFRED.
AI_MODULE	Accès lecture mémoire. Génération réponses. Pas d'admin.	Réservé aux modules IA d'ALFRED.
EMERGENCY	Accès restreint d'urgence. Traçabilité renforcée.	Activation manuelle par OWNER uniquement. Audit obligatoire.

5. Politique de chiffrement et gestion des secrets

Donnée	Méthode de protection
Données persistantes (JSON mémoire)	Fernet (AES 128-CBC + HMAC-SHA256). Clé dans .env.
PIN / Mots de passe	bcrypt rounds=12. Jamais stockés en clair.
Secrets MFA (TOTP)	Chiffrés Fernet avant stockage.
Clés API et tokens	Stockés dans .env. Jamais dans le code. Masqués dans output_filter.
Sessions	Token JWT signé. Expiration 900s. Révocation possible.
Logs sécurité	Stockage local uniquement. Accès ADMIN/OWNER.
Clé Fernet	Générée une seule fois. Stockée dans .env. Rotation planifiée V2.

GESTION DES SECRETS — RÈGLES CRITIQUES

La clé Fernet est générée UNE SEULE FOIS et ne doit JAMAIS être commitée dans Git.

En cas de compromission suspectée de la clé : chiffrement rétroactif obligatoire (data_protection.py → retroactive_encrypt_all).

Rotation des clés planifiée en V2 — à implémenter en priorité critique.

Backup chiffré des clés séparé du backup des données.

6. Conformité aux référentiels

6.1 Couverture OWASP Top 10

Référence	Risque	Mesure ALFRED
A01	Broken Access Control	RBAC 7 rôles + policy_engine + Zero Trust orchestrator
A02	Cryptographic Failures	Fernet + bcrypt + TOTP + clés hors Git
A03	Injection	25 patterns input_validator + threat_detector
A04	Insecure Design	Architecture Zero Trust + fail-safe + defense in depth
A05	Security Misconfiguration	security_governance + dashboard conformité
A06	Vulnerable Components	Tests automatisés + surveillance dépendances
A07	Authentication Failures	bcrypt + MFA + rate limiter + session timeout
A08	Software/Data Integrity	audit_trail JSONL + chiffrement + governance
A09	Logging & Monitoring	security_dashboard + security_logger + audit_trail
A10	SSRF	input_validator : patterns localhost + gopher + LDAP

6.2 Indicateurs de performance sécurité (KPI)

KPI	Objectif
Score sécurité global (dashboard)	≥ 90/100 en permanence
Taux de conformité OWASP	≥ 95%
Couverture tests sécurité	≥ 80% (objectif 100% sur modules critiques)
Temps détection incident	< 30 secondes (alerte dashboard)
Temps réponse P1	< 15 minutes
Disponibilité système	≥ 99% (prototype) → 99.9% (production)
Incidents non résolus > 48h	0 pour P1/P2

7. Gouvernance et responsabilités

Rôle	Responsabilité sécurité
Fondatrice / RSSI (Céline ROUSSELOT)	Responsable ultime de la politique sécurité. Validation des décisions architecturales. Déclenchement P1. Notification RGPD CNIL.
Développeur principal	Application des règles Secure SDLC. Tests sécurité obligatoires. Revue code sécurité. Rapport incidents techniques.

Prestataire externe (développeur Inde)	Respect du CDC Développeur. NDA signé obligatoire. Aucun accès aux données utilisateur réelles. Travail sur environnement de développement isolé.
Développeur Android	Application des règles sécurité Android (Keystore, EncryptedSharedPreferences, SQLCipher). Tests sécurité mobile. Pas d'accès aux secrets backend.

7.2 Revues sécurité périodiques

Fréquence	Activité
Après chaque commit	Vérification automatique : tests, .gitignore, pas de secrets
Hebdomadaire	Revue dashboard sécurité. Incidents ouverts. KPI.
Mensuelle	Audit complet. Revue des rôles actifs. Mise à jour dépendances.
À chaque version majeure	Revue complète politique sécurité. Mise à jour ce document.
En cas d'incident P1/P2	Post-mortem. Revue des mesures. Mise à jour protocole incidents.

8. Roadmap sécurité V2 → V3+

Version	Priorité	Fonctionnalité à implémenter
V2	CRITIQUE	Rotation des clés Fernet — cycle de vie complet
V2	CRITIQUE	Expiration et révocation automatique des appareils
V2	CRITIQUE	Tests de charge sécurité (locust + pytest-benchmark)
V2	CRITIQUE	Corrélation d'événements — détection patterns multi-incidents
V2	ÉLEVÉE	Behavioral detector ML (scikit-learn ou TF Lite local)
V2	ÉLEVÉE	Alerting sécurité temps réel (notification locale)
V2	MOYENNE	Rotation logs + politique de rétention
V3	ÉLEVÉE	SIEM léger local intégré
V3	ÉLEVÉE	IA défensive adaptative
V3	MOYENNE	Sandbox IA — isolation modules
V3+	BASSE	Red Team automatisé
V3+	BASSE	Threat Intelligence (IOC locaux)

ENGAGEMENT DE LA FONDATRICE

Je m'engage personnellement à ce que la sécurité reste la priorité absolue du projet ALFRED à chaque version, chaque décision d'architecture et chaque collaboration.

Cette politique est un document vivant. Elle est mise à jour à chaque version majeure et après chaque incident significatif.

— Céline ROUSSELOT, Fondatrice, Cognitive Products Lab

Cognitive Products Lab · Politique Cybersécurité ALFRED · V1.0 · Mai 2026 · Confidentiel interne